

Scalable Gaussian-Splatting Representation and Real-Time Navigation of Large-Scale Volumetric Electron Microscopy

Draft — Experiments

4. Experiments

This section reports the experiments run against the pipeline described in Section 3. Tables and figures are left blank below, with a red instruction paragraph beneath each one specifying exactly what it should contain and which existing script or output file it should be populated from, so that the authors can insert the final, verified numbers and plots directly.

4.1 Per-Block Reconstruction Fidelity

This experiment establishes whether independent per-block fitting achieves acceptable local reconstruction quality before any cross-block stitching is attempted, and compares the two training budgets described in Section 3.4.

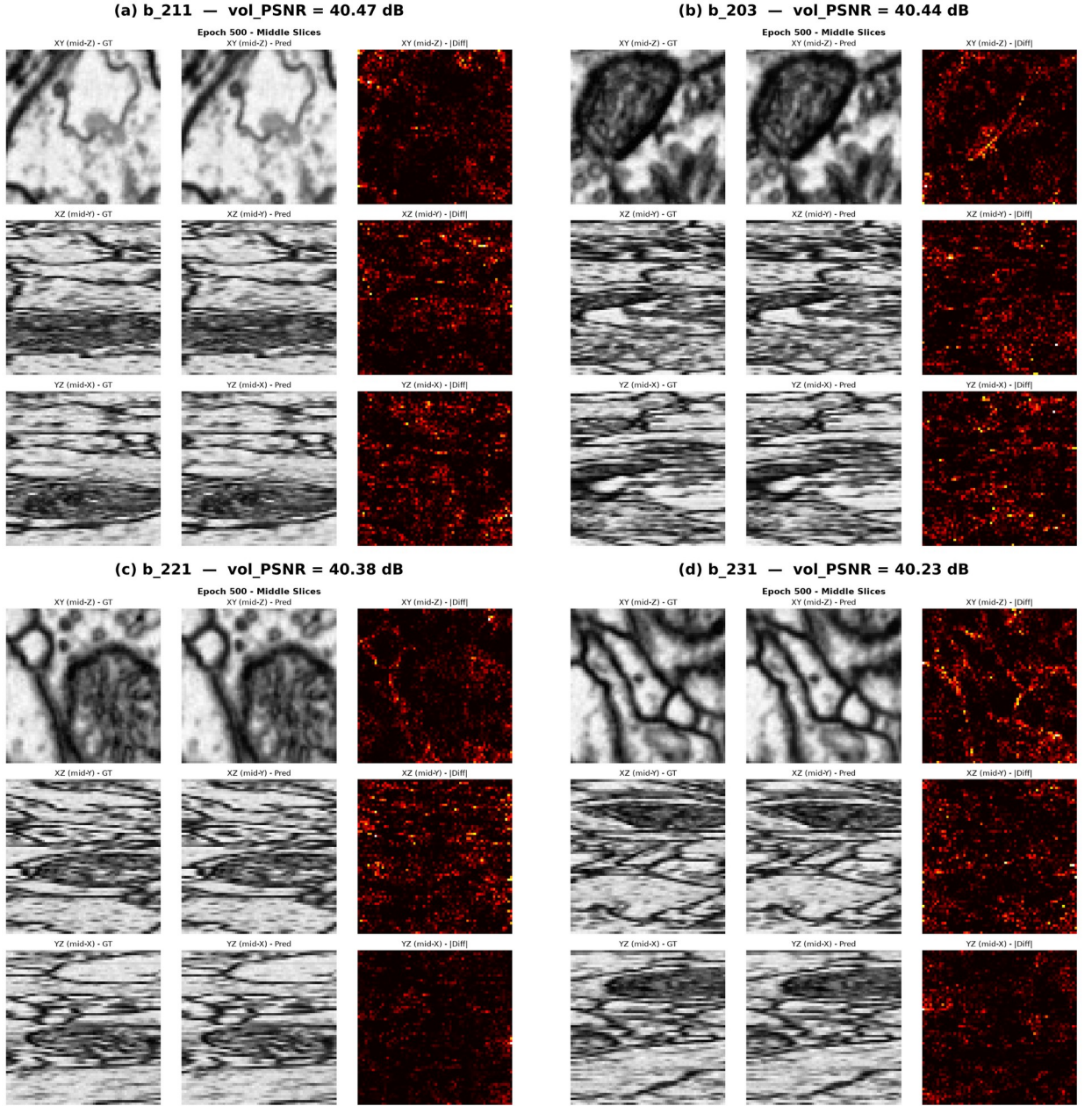
Table 1. Per-block volumetric reconstruction quality (vol_PSNR) across the 64 trained blocks, for each training configuration.

Config	max_gaussians	epochs	Mean vol_PSNR (dB)	Std (dB)	Min (dB)	Max (dB)	Mean time/block
v1	5000	200	20.13	0.74	18.34	21.37	~8.5 min (approx.)
v2	50000	500	39.35	0.50	38.23	40.47	83.6 min
v3	20000	500	33.57	0.44	32.41	34.86	46.8 min

TO FILL IN: populate one row per configuration (v1: config.yml, v2: config_v2.yml) by parsing the ★ new best vol_PSNR lines from each block's models/blocks*/b_*/train.log (already done once earlier in this session — v1 gave mean 20.13 dB / std 0.74, v2 gave mean 39.35 dB / std 0.50 — re-verify and insert current numbers directly from the logs rather than copying those, in case blocks were re-run since).

[insert figure here]

Figure 1. Ground-truth vs. reconstructed middle-slice comparison for the four best-performing v2 (config_v2.yml) blocks: b_211, b_203, b_221, and b_231 (vol_PSNR 40.23-40.47 dB). Each panel shows GT / Prediction / |Diff| for the XY, XZ and YZ mid-planes at the final epoch (500).



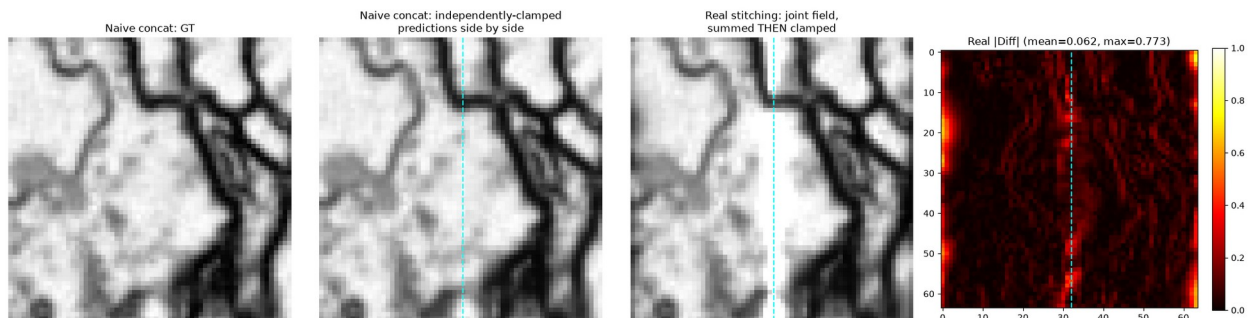
4.2 Cross-Block Stitching Artefact Characterisation

This experiment quantifies the cross-block contamination that arises when independently-trained blocks are combined by the naive additive stitching procedure of Section 3.5. Two adjacent, high-fidelity blocks from the v2 training run (b_211 and b_212, each independently reconstructed to approximately 40.4 dB vol_PSNR in isolation; see Table 1 and Figure 1) were used as a controlled test pair. Each block's trained Gaussian mixture was remapped into a shared coordinate frame spanning both blocks, and a 64^3 voxel cube straddling the shared boundary — the right half

of b_211 and the left half of b_212 — was extracted for direct, ground-truth-referenced evaluation.

As a methodological control, we first verified that reconstructing each block independently and concatenating the resulting voxel grids does not, by itself, reveal any boundary artefact (mean absolute error ≈ 0.006 against ground truth, consistent with ordinary per-block reconstruction noise). This is expected: no interaction between the two blocks' Gaussians occurs in that procedure. The actual stitching pipeline differs in one critical respect — it sums both blocks' raw Gaussian density contributions at every point in the shared frame before evaluation, rather than combining two already-finished reconstructions. Table 2 characterises the effect of this difference directly.

Figure 2. Ground truth vs. two reconstruction procedures at the b_211/b_212 seam (mid-depth slice of the 64^3 evaluation cube; dashed line marks the block boundary): independent per-block reconstruction concatenated after clamping (a negative control, left two panels) vs. the real stitching procedure, which sums both blocks' raw Gaussian densities before clamping (right two panels).

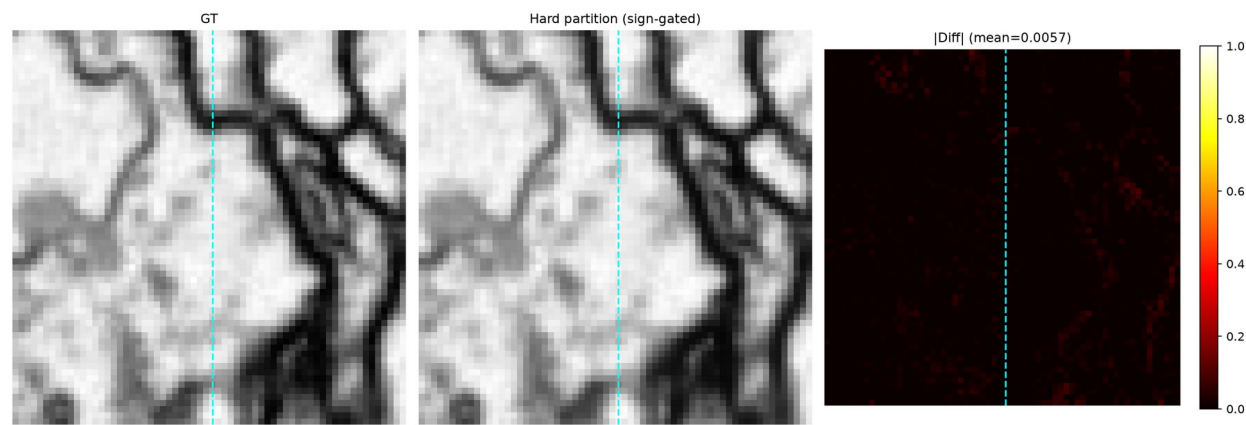


Before quantifying the artefact in Table 2, it is worth showing directly why a naive sanity check can miss it entirely — the two procedures compared here look superficially similar but differ in one critical respect. Panel 1 is ground truth. Panel 2 is the negative control: each block is reconstructed independently from its own Gaussians and clamped to $[0,1]$ individually, and the two finished voxel grids are simply placed side by side. No interaction between the two blocks' Gaussians occurs in this procedure, so the result is, by construction, as clean as each block's own reconstruction — visibly seamless here, with no hint of a boundary. Panel 3 is what the real stitching procedure actually produces: both blocks' raw Gaussians are remapped into one shared coordinate frame and summed at every point before any clamping is applied. The difference is immediately visible — a wide band of saturated (whited-out) voxels spans much of the crop around the boundary, not a thin discontinuity exactly at the seam. Panel 4, the $|\text{difference}|$ map against ground truth for this reconstruction, confirms the effect is both substantial and spatially extensive (mean absolute error 0.062, maximum 0.773 in this particular block pair and evaluation pass), concentrated in the same region where panel 3 saturates.

The two procedures differ only in *when* clamping happens relative to combination — after, for the negative control; before, for the real pipeline — yet only one of them reveals the artefact. A validation check built on the negative-control procedure (panel 2) would wrongly conclude that stitching is artefact-free. This is why every quantitative result in this section (Table 2) evaluates

the pipeline exactly as it is actually used — raw Gaussians summed before clamping — rather than via any procedure that reconstructs and clamps each block independently first.

Figure 2 (continued). Ground truth vs. the hard-partition (sign-gated) reconstruction at the same b_211/b_212 seam and slice as Figure 2, shown on the identical layout and colour scale as the real-stitching panels above, for direct visual comparison.



The merit of the fix is immediately visible against the same ground truth and at the same slice as Figure 2: the hard-partition reconstruction is visually indistinguishable from ground truth, with no trace of the saturated band that dominated the real-stitching panel — and its $|difference|$ map, shown on the same colour scale as Figure 2's, is uniformly near-black rather than showing the bright, spatially extensive error that characterised the naive sum. Table 2a places this side by side with Figure 2's numbers directly, on the same mean/maximum absolute-error scale shown in both figures' colour bars.

Table 2a. Mean and maximum absolute reconstruction error against ground truth for the two procedures shown in Figure 2 / Figure 2 (continued), on the same 64³ seam cube.

Reconstruction	Mean $ Diff $	Max $ Diff $
Real stitching (naive additive sum)	0.062	0.773
Hard partition (sign-gated fix)	0.0057	0.0954

Table 2. Cross-block seam reconstruction quality (vol_PSNR, vol_SSIM, vol_LPIPS) for two density-combination strategies, evaluated against ground truth on a 64³ cube straddling the b_211/b_212 boundary. vol_SSIM and vol_LPIPS follow the same whole-volume convention as vol_PSNR: the respective 2-D metric (skimage SSIM; LPIPS, net='alex') is computed independently on every Z-slice of the cube and averaged, since neither metric has a native 3-D formulation.

Combination strategy	vol_PSNR (dB)	vol_SSIM	vol_LPIPS
Naive additive sum (current production method)	7.64	0.3577	0.3913

Hard partition (sign-gated, block-aware)	39.96	0.9977	0.0012
Reference: each block's native full-volume quality (isolated)	≈ 40.47	n/a	n/a

The naive additive sum — the combination strategy used by the current stitching pipeline — degrades reconstruction quality at the seam by more than 32 dB relative to either block's native quality, with near-total saturation across a wide band around the boundary rather than a thin discontinuity (Figure 2). The mechanism was confirmed directly: each block's Gaussian means are constrained to remain within that block's own volume throughout training (projected back onto the bounding box after every gradient step), but the Gaussian density function itself is not clipped at this boundary, since correctly reconstructing a block's own edge voxels requires Gaussians whose density remains non-negligible beyond it. Evaluated in isolation at the exact seam location, block b_211's Gaussians alone contribute a density of 0.516 and block b_212's alone contribute 0.515 — each an individually plausible reconstruction of that location — while summing them, as the current pipeline does, gives 1.031, saturating after clamping. Both blocks are independently correct; the artefact arises from summing two complete, non-reconciled explanations of the same physical location rather than from either block's training being deficient.

A hard, sign-gated partition was evaluated as a candidate fix — evaluating only the Gaussians belonging to whichever block owns a given point, so that no cross-block summation ever occurs. This recovers 39.96 dB, closing essentially the entire gap to each block's native reconstruction quality (≈ 40.47 dB), and is recommended as the basis for the normalised compositing scheme motivated at the start of this section.

4.3 Reconstruction Quality: Dense Ground Truth vs. Stitched Gaussian Representation

This experiment measures how closely the stitched Gaussian reconstruction matches the dense ground-truth volume, at both the volumetric level and the rendered-MIP level, using the metrics defined in Section 3.9.

Table 3. MIP-level reconstruction quality (dense ground truth vs. stitched Gaussian reconstruction), averaged across the six fixed camera views.

Variant	PSNR (dB)	SSIM	MAE

***TO FILL IN:** run `compute_mip_metrics.py` (or the equivalent aggregate script used earlier in this session) between each dense-GT view render and the matching stitched-Gaussian view render, after independently 1st/99th-percentile normalising both images per Section 3.9, for both the raw and scale-filtered stitched variants; report mean $\pm$ std across the six views, and also*

insert the corresponding per-block volumetric vol_PSNR figure from Table 1 alongside it for direct comparison, since the gap between the two is itself a key finding to discuss.

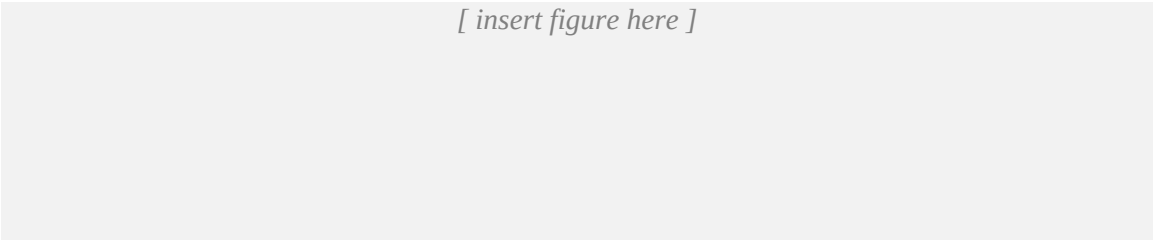


Figure 3. Side-by-side visual comparison of the dense ground-truth MIP render, the raw stitched-Gaussian MIP render, and the scale-filtered stitched-Gaussian MIP render, at one representative camera view.

TO FILL IN: use the same percentile-stretched PNG conversion approach used earlier in this session for visual sanity-checking (pfm_to_png.py with --percentile-low 1 --percentile-high 99), not the raw aligned [0,1] range, since the point of this figure is qualitative structural comparison, not the quantitative units-aligned comparison already reported in Table 3.

4.4 Gaussian Payload vs. Rendering Throughput and Memory

This experiment measures how rendering throughput and memory footprint scale with the number of active Gaussians in the stitched representation, using the nested-subset payload sweep described in Section 3.8.

Table 4. Gaussian rendering throughput and memory footprint as a function of active payload.

Retention	Active Gaussians	Memory (MiB)	Memory / GPU memory (%)	Median FPS

TO FILL IN: re-run the gaussian_mip_rq1_auto_shuffled sweep (scripts/render_scripts/RQ1/) against the current stitched checkpoint and populate one row per retention ratio (10/20/30/40/50/60/75/100%) directly from its output CSV; do not reuse the v1-checkpoint numbers from earlier in this session, since this should reflect whichever checkpoint Table 3 uses.

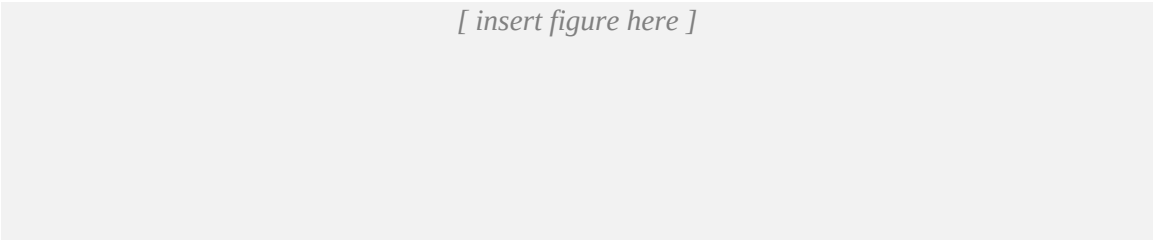


Figure 4. Gaussian rendering FPS vs. active-payload memory footprint (log-log).

TO FILL IN: plot Table 4 directly, log-log axes, one point per retention ratio; this is the Gaussian-only half of the combined Figure 6 below and can reuse the same plotting script.

4.5 Dense-Resident Rendering: FPS vs. Volume Memory Footprint

This experiment measures dense-resident rendering throughput as a function of volume size and memory footprint, independent of any Gaussian representation, at the three assembled volume sizes.

Table 5. Dense-resident rendering throughput and memory footprint by volume size.

Volume size	Memory (MiB)	Memory / GPU memory (%)	Median FPS

TO FILL IN: populate from dense_fps_vs_memory.csv (fafb_pilot/results/), one row per volume size (256³ / 512³ / 1024³); re-verify the FPS figures were measured at the same resolution and depth-sample count as Table 4 before comparing the two (128×128, 64 depth samples was used previously specifically so the two tables would be directly comparable in Figure 6).

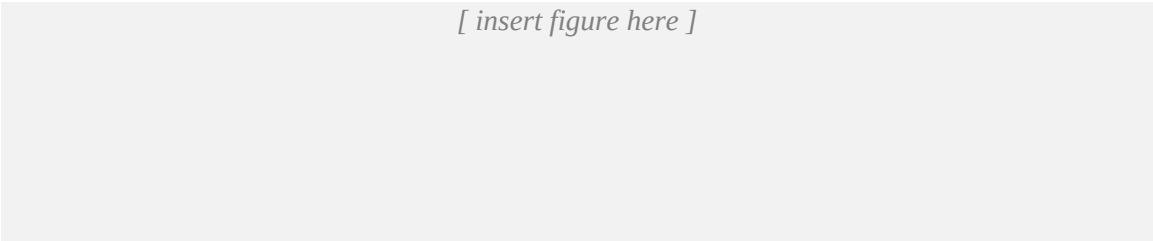


Figure 5. Dense-resident rendering FPS vs. volume memory footprint (log-log).

TO FILL IN: plot Table 5 directly, log-log axes, one point per volume size, labelled 256³/512³/1024³; note in the discussion whether FPS degrades in proportion to memory or faster, since the earlier v1 measurement found a 22× FPS drop from 256³ to 1024³ despite never exceeding 12.5% GPU memory utilisation — re-verify this is still the case.

4.6 Combined FPS vs. Representation Memory Footprint

This is the central comparison of the paper: rendering throughput as a function of memory footprint relative to GPU capacity, for both representations on the same axes, directly instantiating the general metric motivating this work.



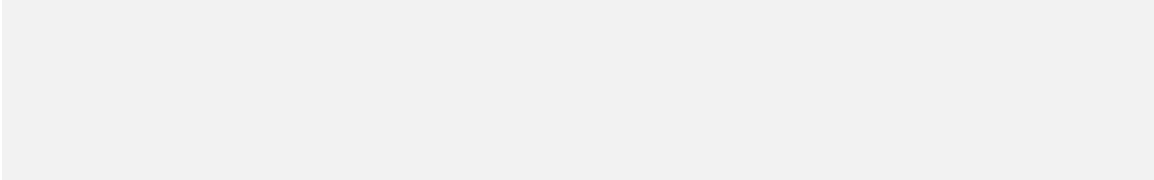


Figure 6. Rendering FPS vs. representation memory / GPU memory (log–log), dense-resident and Gaussian-splatting representations plotted together.

***TO FILL IN:** combine Tables 4 and 5 (or Figures 4 and 5) onto one pair of log–log axes, one series per representation, using two colours from a fixed, validated categorical palette (do not reuse hue for rank); label the dense series' three points by volume size and the Gaussian series' endpoints by active Gaussian count; place the legend in empty plot space rather than over any data; this figure was already produced once earlier in this session (results/figures/fps_vs_memory_ratio.png) and can be regenerated with the same script once Tables 4–5 are confirmed current — the key thing to check when redoing it is whether the roughly 4–5-order-of-magnitude FPS gap between the two series (found previously) still holds, since that is the paper's central empirical claim.*

4.7 Moving-Cube Navigation Benchmark (Proposed)

This experiment evaluates the moving-cube navigation scheme proposed in Section 3.7, measuring throughput and memory as a function of cube size and camera position, for cube sizes with genuine room to move within the currently trained region.

Table 6. Moving-cube rendering throughput and memory footprint, by cube size and camera position, for both representations.

Cube size	Position ID	Representati on	Memory (MiB)	Memory / GPU memory (%)	FPS

***TO FILL IN:** this experiment has not yet been run — it requires the windowed Gaussian-gathering and windowed dense-extraction scripts discussed but not yet built (Section 3.7). Once built, populate one row per (cube size, position, representation) combination; valid grid-aligned positions per cube size within the current 4-blocks-per-axis (256^3) region are 64 for a 1-block cube, 27 for a 2-block cube, and 1 (i.e. no real movement) for a 4-block cube — a genuine 8-block (512^3) cube trial requires training the additional blocks discussed but not yet run.*

[insert figure here]

Figure 7. FPS vs. representation memory / GPU memory, by cube size, with variance across camera positions.

TO FILL IN: aggregate Table 6 to mean $\pm$ std FPS per (cube size, representation) across sampled positions, and plot with error bars on the same log-log axes as Figure 6; this strengthens Figure 6 by showing whether FPS is stable across position for a fixed cube size, or varies enough to be a finding in its own right.

4.8 Downstream-Task Validation (Proposed)

This experiment evaluates whether the stitched Gaussian reconstruction preserves the structure needed for a real downstream connectomics task, rather than relying solely on the pixel-level metrics in Section 4.3, following the validation strategy discussed in Section 2.2 for the closest related work.

Table 7. Downstream structural-fidelity metric (ground truth vs. stitched Gaussian reconstruction), definition to be finalised.

Metric	Ground truth	Raw stitched	Filtered stitched

TO FILL IN: this experiment has not yet been run or fully designed. The segment_.tif instance-segmentation labels already available for this dataset, together with the existing scripts/data_scripts/find_neuron_voxels.py crawler, are the natural starting point: voxelise the reconstructed Gaussian field back onto the ground-truth grid and measure how well intensity/structure boundaries agree with the real segment boundaries (e.g. a boundary-agreement or IoU-style score), rather than reporting raw per-voxel intensity error.*

[insert figure here]

Figure 8. Example overlay of ground-truth segment boundaries on the dense and stitched-Gaussian reconstructions, for one representative region.

TO FILL IN: once Table 7's metric is defined, produce one qualitative example showing segment boundaries (from `segment_*.tif`) overlaid on both the dense GT slice and the corresponding stitched-Gaussian reconstruction slice, so a reader can visually judge what the quantitative score in Table 7 corresponds to.